

Лабораторная работа № 3.

Интеграционное тестирование REST API

Тема: Supertest, Express integration tests, JWT и test database

Цель: Настроить integration testing REST API Internet Shop через Jest и Supertest, отделить Express app от app.listen(), проверить HTTP status/body/headers, protected endpoints и работу с отдельной test database.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте проект express-shop. Должны работать Jest, Product API, JWT Authentication/Authorization, Prisma и PostgreSQL. В этой лабораторной используется отдельная test database.

1. Установите Supertest

PowerShell

```
npm install --save-dev supertest
```

2. Отделите app от server

app.js

```
import express from "express";
import productRoutes from "../routes/productRoutes.js";
import authRoutes from "../routes/authRoutes.js";

export const app = express();

app.use(express.json({
  limit: "100kb"
}));

app.use("/api/auth", authRoutes);
app.use("/api/products", productRoutes);
```

server.js

```
import { app } from "../app.js";

const PORT = process.env.PORT ?? 3000;

app.listen(PORT, () => {
  console.log(`Server started on port ${PORT}`);
});
```

В integration tests импортируется app. Запуск app.listen() выполняется только в server.js.

3. Создайте test environment

```
.env.test

NODE_ENV=test
DATABASE_URL=postgresql://shop_test:college123@localhost:5432/shop_test
JWT_SECRET=test-only-secret
```

Не используйте development или production database для integration tests.

4. Создайте test database

```
PostgreSQL

CREATE DATABASE shop_test OWNER shop_user;
```

Если у вас используется отдельный test database user, укажите его в DATABASE_URL.

5. Примените migrations к test database

```
PowerShell

$env:DATABASE_URL="postgresql://shop_test:college123@localhost:5432/shop_test"
npx prisma@7 migrate deploy
```

Test database должна иметь ту же schema, что и приложение, но отдельные test data.

6. Создайте integration test file

```
Структура проекта

tests/
  unit/
    orderMath.test.js
    productService.test.js
  integration/
    products.api.test.js
```

7. Подключите Supertest и Jest

```
tests/integration/products.api.test.js

import request from "supertest";
import {
  beforeEach,
  describe,
  expect,
  test
} from "@jest/globals";
import { app } from "../../app.js";
import { prisma } from "../../config/prisma.js";
```

8. Очищайте test data перед каждым test

```
tests/integration/products.api.test.js

beforeEach(async () => {
  await prisma.orderItem.deleteMany();
  await prisma.order.deleteMany();
  await prisma.product.deleteMany();
  await prisma.category.deleteMany();

  await prisma.category.create({
    data: {
      id: 1,
      name: "Accessories"
    }
  });
});
```

Порядок удаления должен учитывать внешние ключи.

9. Создайте первый GET integration test

```
tests/integration/products.api.test.js

describe("Product API", () => {
  test("GET /api/products returns products", async () => {
    await prisma.product.create({
      data: {
        name: "Mouse",
        price: 12000,
        categoryId: 1,
        inStock: true
      }
    });

    const response = await request(app)
      .get("/api/products");

    expect(response.status).toBe(200);
    expect(Array.isArray(response.body)).toBe(true);
    expect(response.body).toHaveLength(1);
  });
});
```

10. Проверьте Content-Type

Дополнение к GET test

```
expect(response.headers["content-type"])
  .toMatch(/json/);
```

11. Проверьте GET /api/products/:id

GET Product by id

```
test("GET /api/products/:id returns product", async () => {
```

```

const product = await prisma.product.create({
  data: {
    name: "Keyboard",
    price: 25000,
    categoryId: 1,
    inStock: true
  }
});

const response = await request(app)
  .get(`/api/products/${product.id}`);

expect(response.status).toBe(200);
expect(response.body).toEqual(
  expect.objectContaining({
    id: product.id,
    name: "Keyboard",
    price: 25000
  })
);
});

```

12. Проверьте 404

GET unknown Product

```

test("GET unknown product returns 404", async () => {
  const response = await request(app)
    .get("/api/products/999999");

  expect(response.status).toBe(404);
  expect(response.body.error)
    .toBe("Product not found");
});

```

13. Подготовьте admin user

Setup admin

```

import argon2 from "argon2";

async function createAdminUser() {
  const passwordHash = await argon2.hash(
    "TestPassword123!"
  );

  return prisma.user.create({
    data: {
      email: "admin@test.local",
      passwordHash,
      role: "admin"
    }
  });
}

```

14. Получите JWT через настоящий login endpoint

Login helper

```
async function loginAsAdmin() {
  await createAdminUser();

  const response = await request(app)
    .post("/api/auth/login")
    .send({
      email: "admin@test.local",
      password: "TestPassword123!"
    });

  expect(response.status).toBe(200);

  return response.body.accessToken;
}
```

15. Проверьте POST без token

401 Unauthorized

```
test("POST /api/products without token returns 401", async () => {
  const response = await request(app)
    .post("/api/products")
    .send({
      name: "USB Hub",
      price: 12000,
      categoryId: 1,
      inStock: true
    });

  expect(response.status).toBe(401);
});
```

16. Проверьте POST с admin token

201 Created

```
test("admin creates product", async () => {
  const adminToken = await loginAsAdmin();

  const response = await request(app)
    .post("/api/products")
    .set("Authorization", `Bearer ${adminToken}`)
    .send({
      name: "USB Hub",
      price: 12000,
      categoryId: 1,
      inStock: true
    });

  expect(response.status).toBe(201);
  expect(response.body).toEqual(
    expect.objectContaining({
      name: "USB Hub",
      price: 12000
    })
  );
});
```

```
    })  
  );  
});
```

17. Проверьте запись в test database

DB assertion

```
const stored = await prisma.product.findFirst({  
  where: {  
    name: "USB Hub"  
  }  
});  
  
expect(stored).not.toBeNull();  
expect(stored.price).toBe(12000);
```

Integration test может проверять не только HTTP response, но и результат интеграции с реальной test database.

18. Проверьте validation error

400 Validation failed

```
test("invalid product returns 400", async () => {  
  const adminToken = await loginAsAdmin();  
  
  const response = await request(app)  
    .post("/api/products")  
    .set("Authorization", `Bearer ${adminToken}`)  
    .send({  
      name: "X",  
      price: 0,  
      categoryId: 1  
    });  
  
  expect(response.status).toBe(400);  
  expect(response.body.error)  
    .toBe("Validation failed");  
});
```

19. Проверьте PATCH

PATCH Product

```
test("admin updates product", async () => {  
  const adminToken = await loginAsAdmin();  
  
  const product = await prisma.product.create({  
    data: {  
      name: "Old name",  
      price: 10000,  
      categoryId: 1,  
      inStock: true  
    }  
  });  
});
```

```
const response = await request(app)
  .patch(`/api/products/${product.id}`)
  .set("Authorization", `Bearer ${adminToken}`)
  .send({
    name: "New name"
  });

expect(response.status).toBe(200);
expect(response.body.name).toBe("New name");
});
```

20. Проверьте DELETE

DELETE Product

```
test("admin deletes product", async () => {
  const adminToken = await loginAsAdmin();

  const product = await prisma.product.create({
    data: {
      name: "Temporary",
      price: 5000,
      categoryId: 1,
      inStock: true
    }
  });

  const response = await request(app)
    .delete(`/api/products/${product.id}`)
    .set("Authorization", `Bearer ${adminToken}`);

  expect([200, 204]).toContain(response.status);

  const stored = await prisma.product.findUnique({
    where: { id: product.id }
  });

  expect(stored).toBeNull();
});
```

21. Проверьте user role → 403

Создайте отдельного пользователя с ролью `user`, выполните login и попробуйте вызвать административный POST `/api/products`.

Ожидаемая проверка

```
expect(response.status).toBe(403);
```

22. Запустите integration suite

PowerShell

```
npm test -- tests/integration/products.api.test.js --runInBand
```

`--runInBand` полезен для учебного integration suite с одной общей test database, потому что tests выполняются последовательно.

23. Самостоятельное задание

1. Добавьте test GET `/api/products` для пустой базы.
2. Добавьте test для invalid id в URL.
3. Добавьте test POST с обычным user token и ожидаемым 403.
4. Добавьте test для invalid JWT и ожидаемого 401.
5. Добавьте test для PATCH неизвестного Product.
6. Добавьте test для DELETE неизвестного Product.
7. После POST проверьте запись через Prisma.
8. После DELETE проверьте отсутствие записи.
9. Убедитесь, что каждый test можно запускать отдельно.
10. Запустите suite два раза подряд и получите одинаковый результат.

Требования к результату

- Supertest установлен как dev dependency.
- app.js не вызывает app.listen().
- server.js запускает приложение.
- Integration tests используют request(app).
- Есть отдельная test database.
- Test data очищается и создаётся заново.
- Проверяется GET collection.
- Проверяется GET resource.
- Проверяется 404.
- Проверяется 401.
- Проверяется 403.
- Проверяется POST 201.
- Проверяется validation 400.
- Есть PATCH и DELETE tests.
- После mutations проверяется состояние test database.

Критерии оценивания

№	Критерий
1	Supertest установлен
2	app и app.listen() разделены
3	Используется request(app)
4	GET collection возвращает 200
5	GET resource возвращает 200
6	Неизвестный Product возвращает 404
7	POST без token возвращает 401

№	Критерий
8	User без admin role получает 403
9	Admin POST возвращает 201
10	Invalid body возвращает 400
11	PATCH endpoint протестирован
12	DELETE endpoint протестирован
13	Используется отдельная test database
14	Setup/cleanup обеспечивает isolation
15	Студент объясняет отличие unit, integration и E2E testing